SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I M.harini(192424011) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: M.harini

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode– execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing

modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	**Understanding of Problem Context (20%)**	**Application of Concepts (20%)**	**Problem-Solving Approach (20%)**	**Accuracy of Solution (20%)**	**Clarity (20%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. Smart City Surveillance System – CPU, Memory, I/O and Bus Structure

Introduction

A Smart City Surveillance System is an intelligent security solution used to monitor public places such as roads, airports, railway stations, shopping malls, schools, hospitals, and government offices. It consists of hundreds or even thousands of CCTV cameras that continuously capture live video and transmit it to a central computer system for processing. The system uses Artificial Intelligence (AI), Machine Learning (ML), and Image Processing techniques to detect suspicious activities such as theft, accidents, traffic violations, abandoned objects, and unauthorized entry.

The surveillance system depends on three important hardware components: the CPU, Memory, and Input/Output (I/O) devices. Cameras act as input devices that continuously send video data to the computer. The memory stores these video frames temporarily, while the CPU fetches the data from memory and executes image processing algorithms to identify threats. Once a threat is detected, the system immediately sends alerts to security personnel.

During peak hours, a large number of cameras generate huge amounts of video data every second. This increases the workload on the CPU, memory, and communication bus. If the system cannot transfer and process the data efficiently, delays occur in threat detection. Therefore, an efficient bus architecture and faster instruction execution techniques are essential for maintaining real-time performance.

Objectives

- To understand the interaction between CPU, Memory, and I/O devices.
- To analyze the reasons for system lag during peak hours.
- To compare Single-Bus and Multi-Bus architectures.
- To study the instruction execution cycle.
- To identify methods for improving overall system performance.

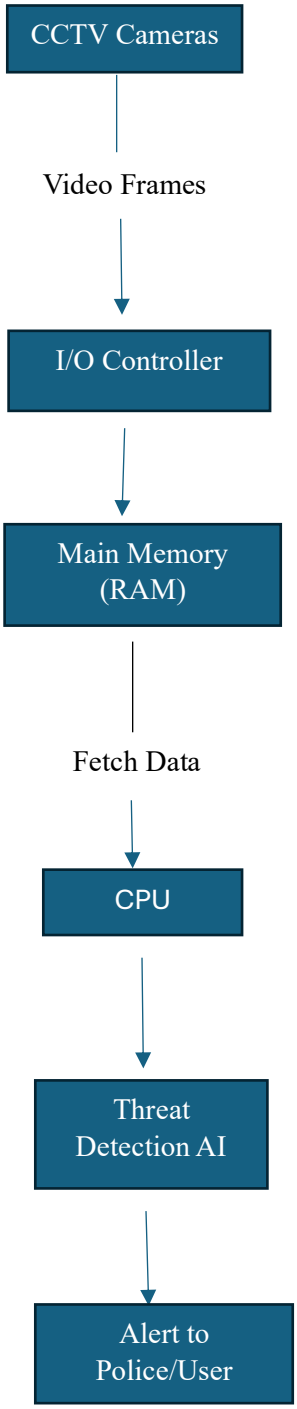
Applications

Smart City Surveillance Systems are widely used in various fields, including:

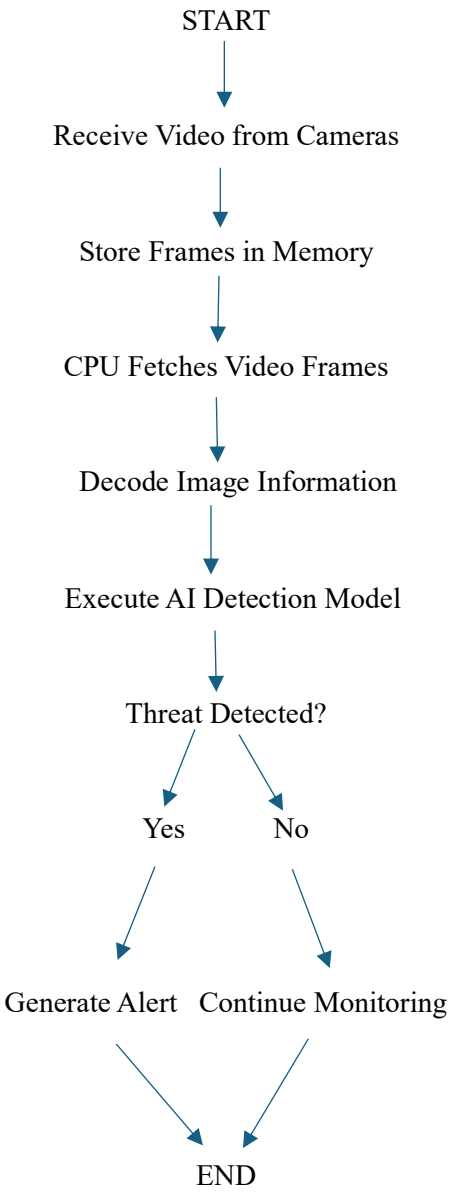
- Traffic monitoring and accident detection.
- Crime prevention and investigation.
- Airport and railway station security.
- Smart parking management.
- Crowd monitoring during public events.
- Border and military surveillance.
- Shopping mall security.
- Industrial safety monitoring.

- Smart campus surveillance.
- Disaster management and emergency response.

System Architecture



FLOWCHART



C Program

```
#include <stdio.h>

int main()

{

   int cameras, i;

   printf("Enter number of cameras: ");

   scanf("%d", &cameras);

   for(i = 1; i <= cameras; i++)

   {

      printf("\nReceiving video from Camera %d", i);
```

```
        printf("\nStoring frame in Memory");
        printf("\nCPU Processing Frame");
        printf("\nThreat Detection Completed\n");
    }
    printf("\nAll camera feeds processed successfully.");
    return 0;
}
```

Limitations

- High installation cost.
- Large storage requirement.
- High power consumption.
- Network congestion during peak hours.
- Privacy concerns.
- Requires regular maintenance.

Conclusion

A Smart City Surveillance System relies on the efficient interaction of the CPU, memory, and I/O devices to process large volumes of video data in real time. During peak hours, delays occur because of bus congestion, increased memory access, and CPU waiting time. A multi-bus architecture, faster memory, cache optimization, pipelining, DMA, and multi-core processors significantly improve system performance and reduce threat detection delays. By adopting these architectural enhancements, smart cities can provide faster, more reliable, and secure surveillance services.

2. Real-Time Stock Trading Application – Fetch, Decode, Execute Cycle and CPI

Introduction

A Real-Time Stock Trading System is a computer-based application that allows investors to buy and sell shares instantly through stock exchanges. Every transaction is processed by the processor in real time, where millions of instructions are executed every second. The CPU must continuously fetch instructions, decode them, execute calculations, access memory, and update the transaction database.

During normal conditions, the processor can complete transactions within milliseconds. However, during periods of heavy trading such as company earnings announcements, IPO launches, or market crashes, millions of users place buy and sell orders simultaneously. This increases CPU workload, memory access, and branch instructions, leading to a high Cycles Per Instruction (CPI).

A higher CPI means the processor requires more clock cycles to execute each instruction, increasing execution time and causing transaction delays. Modern processors overcome these issues using cache memory, pipelining, branch prediction, superscalar execution, and multicore architecture to ensure high-speed processing.

Objectives

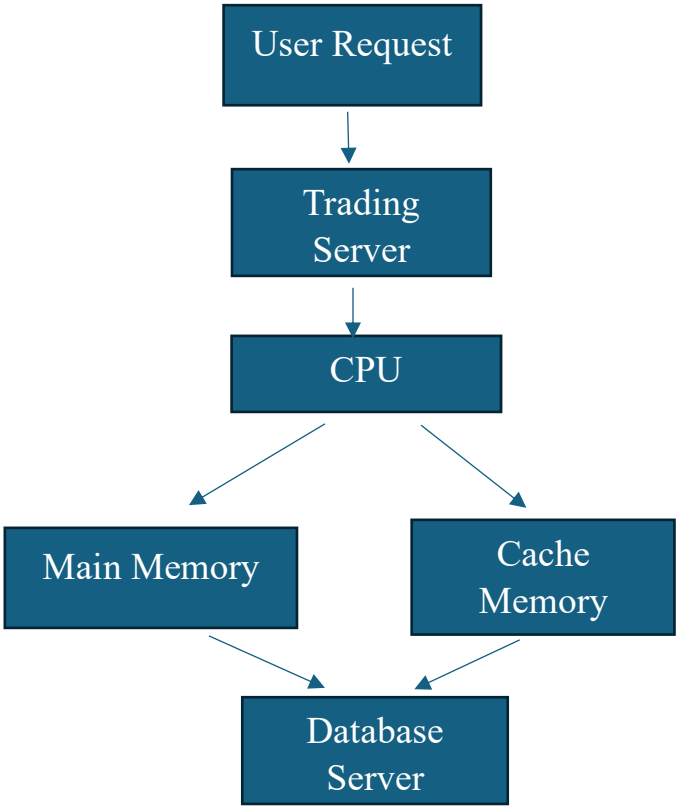
- To understand the Fetch–Decode–Execute cycle.
- To analyze the effect of CPI on processor performance.
- To identify the causes of delays in stock trading systems.
- To compare different CPU architectural improvements.
- To study techniques used for reducing execution time.

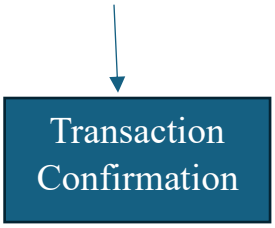
Applications

Real-time stock trading systems are used in:

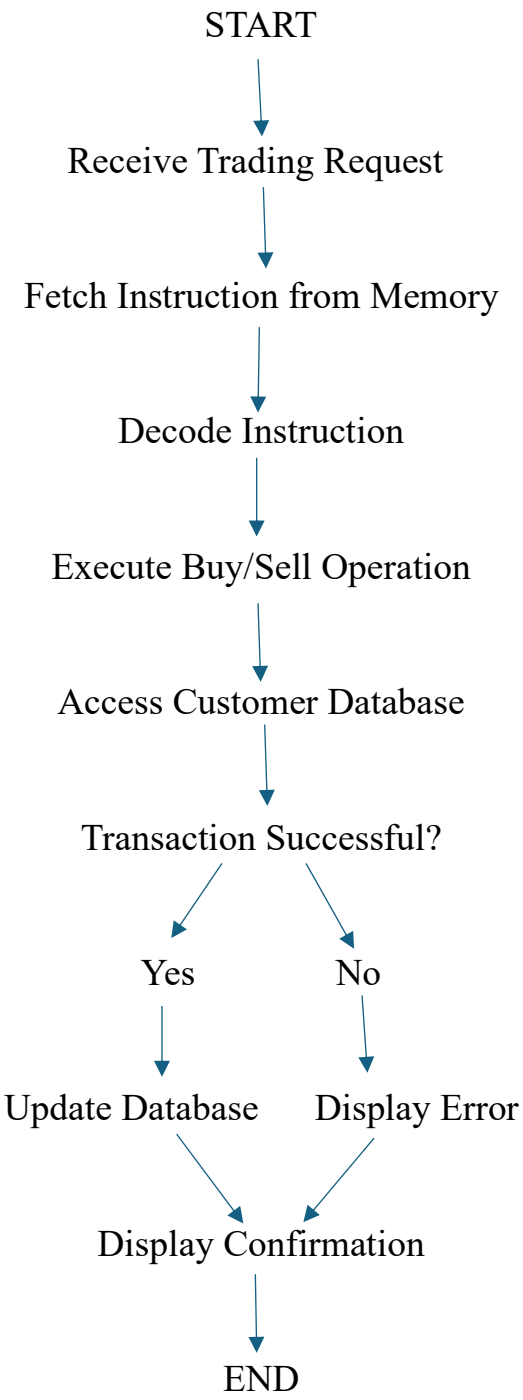
- Online Stock Exchanges
- Banking Systems
- Financial Institutions
- Cryptocurrency Trading Platforms
- Online Payment Gateways
- Insurance Companies
- Investment Firms
- High Frequency Trading (HFT)
- Mutual Fund Management
- Financial Data Analytics

System Architecture





Flowchart



C PROGRAM

```
#include <stdio.h>

int main()

{

   int amount = 1000;
```

```
    int balance = 5000;
    if (balance >= amount)
    {
        balance = balance - amount;
        printf("Transaction Successful\n");
    }
    else
    {
        printf("Transaction Failed\n");
    }
    printf("Remaining Balance = %d", balance);
    return 0;
}
```

Limitations

- High hardware cost.
- Heavy memory usage.
- Network congestion during peak hours.
- High CPI may reduce performance.
- Requires reliable internet connection.
- Complex system maintenance.

Conclusion

A real-time stock trading application depends on the efficient execution of millions of processor instructions every second. During high market activity, frequent memory accesses, cache misses, and branch instructions increase the Cycles Per Instruction (CPI), causing delays in transaction processing. The Fetch–Decode–Execute cycle directly affects execution time because every transaction must pass through these stages before completion. Modern architectural techniques such as cache memory, instruction pipelining, branch prediction, superscalar execution, out-of-order execution, multicore processors, and faster memory technologies significantly reduce CPI and improve CPU performance. These enhancements ensure that trading systems remain fast, reliable, and capable of handling high transaction volumes with minimal delay.

3. Embedded Temperature Control System using 8085 Microprocessor

Introduction

An embedded system is a specialized computer system designed to perform a specific task efficiently. In industries, embedded systems are widely used for temperature monitoring and control to ensure machines operate safely. The 8085 microprocessor acts as the controller, receiving temperature values from sensors, processing the data, and controlling actuators such as cooling fans, heaters, or alarms.

The 8085 microprocessor uses different addressing modes to access data from registers, memory, or input/output devices. If the wrong addressing mode is used, the processor may read incorrect sensor values or store data in the wrong memory location. This can lead to inaccurate temperature control, equipment damage, or production failures. Therefore, selecting the correct addressing mode and instruction set is essential for reliable system operation.

Objectives

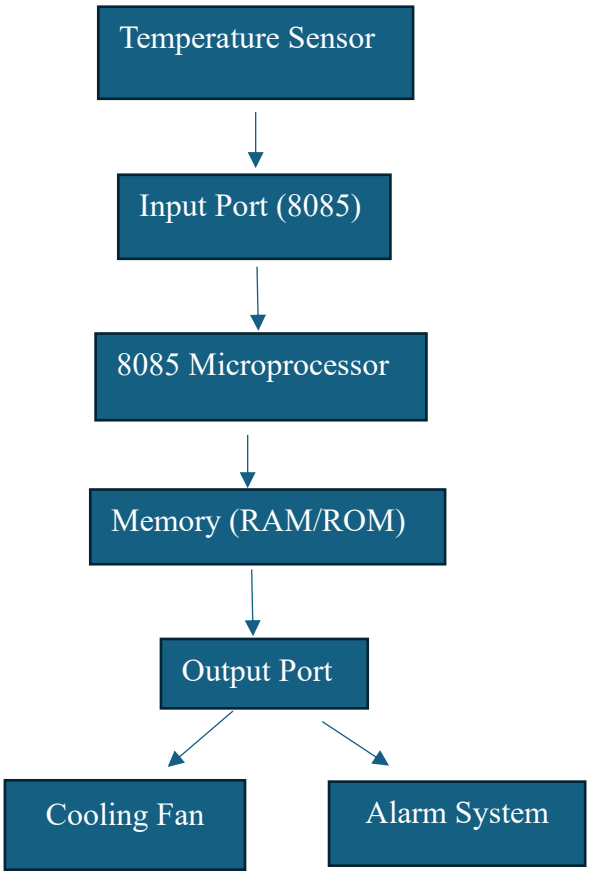
- To understand the role of the 8085 microprocessor in an embedded system.
- To study different addressing modes of the 8085.
- To analyze causes of incorrect temperature readings.
- To understand how the 8085 instruction set ensures accurate operation.
- To improve the reliability of industrial temperature control systems.

Applications

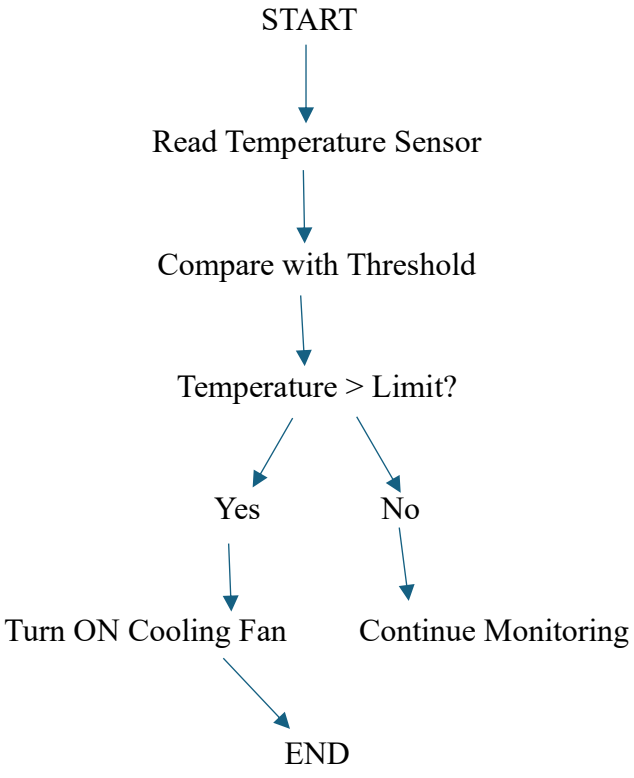
The 8085-based temperature control system is used in:

- Industrial furnace control
- Air conditioning systems
- Refrigeration plants
- Boiler temperature monitoring
- Chemical industries
- Food processing industries
- Pharmaceutical manufacturing
- Power plants
- Smart home automation
- Laboratory equipment

System Architecture



Flowchart



C PROGRAM

```
#include<stdio.h>
int main()
{
   int temp;
   printf("Enter Temperature: ");
   scanf("%d",&temp);
   if(temp>50)
      printf("Cooling Fan ON");
   else
      printf("Temperature Normal");
   return 0;
}
```

Limitations

- Limited memory in the 8085.
- Slower than modern microprocessors.
- Cannot process complex applications.
- Requires correct programming.
- Limited multitasking capability.

Conclusion

The 8085 microprocessor is widely used in industrial temperature control systems because of its simple architecture and reliable operation. The choice of addressing mode plays an important role in accessing sensor data correctly. Improper addressing or incorrect instructions can lead to inaccurate temperature readings and unsafe system behavior. By using the appropriate addressing modes, 8085 instruction set, and proper programming techniques, the system can accurately monitor temperature, control cooling devices, and ensure safe and efficient industrial operation.